

Modelagem de Problema de Otimização Combinatória

Raiane Gabriele Brito de Oliveira

¹Universidade Federal do Pará
Instituto de Ciências Exatas e Naturais
Bacharelado em Ciência da Computação
Belém – PA – Brasil

raiane.oliveira@icen.ufpa.br

Sobre o Trabalho

O problema envolve a implementação de uma heurística para encontrar o menor custo total na alocação de 9 projetos entre 9 empresas. O objetivo é minimizar o custo total, assegurando que cada empresa seja responsável por exatamente um projeto e que nenhum projeto seja repetido. Ou seja, trata-se de um problema de otimização combinatória com restrição de atribuição única.

Este trabalho utiliza o método heurístico Hill Climbing com Multi-start, que busca iterativamente soluções melhores a partir de uma solução inicial aleatória, sempre aceitando apenas vizinhos que melhorem o custo total. A implementação foi feita em linguagem Python e retorna uma solução viável em tempo aceitável.

Como a solução é codificada:

A solução é representada por uma lista de inteiros de 1 a 9, onde cada elemento representa o projeto alocado à respectiva empresa. Por exemplo, a solução [3, 1, 4, 2, 6, 5, 9, 8, 7] indica que a empresa 0 ficou com o projeto 3, a empresa 1 com o projeto 1, e assim por diante.

Essa estrutura permite uma codificação simples, pois cada posição da lista corresponde diretamente a uma empresa, e o valor nessa posição indica o projeto atribuído.

Como a solução inicial é gerada:

A solução inicial é gerada de forma aleatória por meio do embaralhamento (função shuffle) de uma lista contendo os números de 1 a 9. Isso garante que todos os projetos estejam presentes na solução inicial, respeitando a restrição de que cada projeto só pode ser alocado uma vez.

Essa abordagem assegura que a solução inicial já seja viável, embora provavelmente longe do ótimo.

Qual a função de vizinhança utilizada:

A função de vizinhança utilizada consiste na troca de projetos entre duas empresas. Para cada par de empresas (i, j) , gera-se um novo vizinho ao trocar os projetos alocados entre essas duas empresas. O novo custo é calculado e, caso seja menor que o custo atual, o vizinho é considerado uma melhoria.

Dessa forma, o espaço de busca é explorado apenas em direções que resultam em soluções melhores, caracterizando um *Hill Climbing* estrito, que não aceita piores soluções.

Quais critérios são utilizados para realização da busca:

A busca segue os seguintes critérios:

- Inicializa-se com uma solução viável aleatória;
- Gera-se todos os vizinhos que apresentam custo menor que o atual;
- Dentre os vizinhos melhores, seleciona-se o de menor custo;
- Atualiza-se a solução atual com o melhor vizinho;
- O processo se repete até que não existam vizinhos com custo menor que o atual (mínimo local).

Para evitar ficar preso em mínimos locais, é utilizada a estratégia de multi-start, em que o algoritmo é executado múltiplas vezes ($n = 120$ na execução final) com diferentes soluções iniciais, e ao final retorna-se a melhor solução encontrada entre todas.

A solução final é apresentada tanto em forma de lista (projeto por empresa) quanto em uma matriz binária de alocação (empresa x projeto). Além disso, o gráfico com a evolução dos custos de todas as execuções é exibido com destaque para a melhor delas, permitindo uma análise visual do comportamento da heurística.

Código Fonte

```
import random
import matplotlib.pyplot as plt
import time

inicio = time.time()
matrix_valores = [
    [12, 18, 15, 22, 9, 14, 20, 11, 17],
    [19, 8, 13, 25, 16, 10, 7, 21, 24],
    [6, 14, 27, 10, 12, 19, 23, 16, 8],
    [17, 11, 20, 9, 18, 13, 25, 14, 22],
    [10, 23, 16, 14, 7, 21, 12, 19, 15],
    [13, 25, 9, 17, 11, 8, 16, 22, 20],
    [21, 16, 24, 12, 20, 15, 9, 18, 10],
    [8, 19, 11, 16, 22, 17, 14, 10, 13],
    [15, 10, 18, 21, 13, 12, 22, 9, 16]
]

def calcular_custo(solucao):
    custo_total = 0
    for empresa, projeto in enumerate(solucao):
        custo_total += matrix_valores[empresa][projeto - 1]
    return custo_total

def gerar_vizinhos_melhores(solucao, custo_atual):
    vizinhos = []
    for i in range(len(solucao)):
        for j in range(i + 1, len(solucao)):
```

```

        vizinho = solucao.copy()
        vizinho[i], vizinho[j] = vizinho[j], vizinho[i]
        custo_vizinho = calcular_custo(vizinho)
        if custo_vizinho < custo_atual:
            vizinhos.append((vizinho, custo_vizinho))
    return vizinhos

def hill_climbing():
    solucao_atual = list(range(1, 10))
    random.shuffle(solucao_atual)
    custo_atual = calcular_custo(solucao_atual)
    custos = [custo_atual]

    while True:
        vizinhos_melhores = gerar_vizinhos_melhores(solucao_atual,
            custo_atual)
        if not vizinhos_melhores:
            break

        melhor_vizinho, melhor_custo = min(vizinhos_melhores, key=
            lambda x: x[1])
        solucao_atual = melhor_vizinho
        custo_atual = melhor_custo
        custos.append(custo_atual)

    return solucao_atual, custo_atual, custos

def multi_start_hill_climbing(n):
    melhor_solucao = None
    melhor_custo = float('inf')
    melhor_custos = []
    todas_execucoes = []

    for _ in range(n):
        solucao, custo, custos = hill_climbing()
        todas_execucoes.append(custos)
        if custo < melhor_custo:
            melhor_solucao = solucao
            melhor_custo = custo
            melhor_custos = custos

    return melhor_solucao, melhor_custo, melhor_custos, todas_execucoes

solucao, custo, melhor_custos, todas_execucoes =
    multi_start_hill_climbing(120)

print("Melhor_solução_(projeto_por_empresa):", solucao)
print("Custo_total:", custo)

print("\nMatriz_binária_(empresa_x_projeto):")
matriz_binaria = [[0 for _ in range(9)] for _ in range(9)]
for empresa, projeto in enumerate(solucao):
    matriz_binaria[empresa][projeto - 1] = 1

for linha in matriz_binaria:
    print(' '.join(str(valor) for valor in linha))

```

```

fim = time.time()
print(f"Tempo total de execução: {fim - inicio:.2f} segundos")

plt.figure(figsize=(15, 7.5))
for i, execucao in enumerate(todas_execucoes):
    plt.plot(execucao, alpha=0.4)
plt.plot(melhor_custos, color='red', linewidth=2.5, label='Melhor execução')
plt.title('Evolução dos custos em todas as execuções do Hill Climbing')
plt.xlabel('Iterações')
plt.ylabel('Custo')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```